CodeSense

A research prototype for learning to *read, judge and shape* code in a generative-AI-rich classroom · Computing Science dissertation project · Django · offline

PURPOSE

Introductory programming students now receive code from AI tools at least as often as they write it. CodeSense is a small web app for studying how to teach the skills that matter in that setting: understanding what code does, finding why it fails, and deciding whether generated code can be trusted. It is a research instrument, not a general practice site: educational design and evaluation come before engineering complexity.

WHAT A STUDENT DOES

Students pick one of six topics (Variables, Conditionals, Loops, Functions, Lists, Dictionaries) and work through short activities. There are no accounts. Each activity shows a small piece of code and one question; the student can ask for a hint, then submit and receive feedback with an expert explanation, the misconceptions the item targets, and a follow-up idea. The app has **48 activities** across **16 activity types**.

EDUCATIONAL FRAMEWORK (THREE LAYERS, EACH ADDITIVE)

- **Many Small Problems.** One concept per item, short and low-pressure, with quick feedback.
- **Bloom-inspired outcomes (L1–L6).** Every topic has activities at every level, and each level is a different cognitive task rather than a harder version of the same question: recall vocabulary, trace code, complete or modify code, analyse faults, extend or improve code, evaluate code (including AI output).
- **Block Model of Program Comprehension.** Each activity is also tagged with the *grain* of program it concerns: Atoms (syntax elements), Blocks (a loop, conditional or function), Relations (data and control flow), Macro Structure (the program's purpose). Bloom says how demanding a task is; the Block Model says what it is about. This is metadata only and does not gate progress.

Bloom \ Block	Atoms	Blocks	Relations	Macro	Typical activity types
L1 Knowledge	15	–	–	–	Identify terminology, define a concept, match terms to definitions
L2 Comprehension	–	4	3	2	Explain code, trace, program purpose
L3 Application	1	4	–	1	Complete the code, modify the code, prompt construction
L4 Analysis	–	–	6	–	Debug, compare solutions, analyse a misconception
L5 Synthesis	–	–	–	6	Extend the code, improve / refactor
L6 Evaluation	–	–	3	3	Judge an AI suggestion, review code quality

Reading, tracing, debugging and judging make up about four fifths of the activities; writing tasks are the minority, and even those accept a description in words as well as code.

ROLE OF AI

AI scaffolds and never solves the activity. It supplies hints, alternative explanations and follow-ups, and a set of activities asks students to *accept or reject* AI-generated code with reasons (some are correct, so rejecting everything does not pay). Options are built from documented novice misconceptions rather than random wrong answers. AI sits behind an abstraction (`AIService`) with a deterministic mock, so no API key is needed.

FEEDBACK AND ASSESSMENT

- Multiple choice, matching (scored per pair) and accept/reject are checked deterministically.
- Open answers are **formative, never graded**: a pluggable `FeedbackService` returns the ideas found and missing, a score and feedback text.
- The default assessor matches keywords (transparent and free). An AI-enhanced assessor returns the same result shape and, without a key, uses a mocked response; bad model output falls back to keywords.

RESEARCH DATA AND PRIVACY

- Participants are random codes (e.g. `P-3F9A12C4`). No names, emails, IP addresses, user agents or session keys are stored.
- Activity data (the `practice` app) is kept apart from evaluation data (the `research` app), so either can be exported, anonymised or deleted separately.
- Analytics, answer-text storage, hint counts and timestamps can each be switched off in settings; the app works identically when analytics are off.
- Attempts export to CSV with topic, activity type, Bloom and Block level, correctness, score and concepts found or missing.

TECHNOLOGY

Django 5.2, SQLite and Django templates; Bootstrap, fonts and CSS are bundled locally, so it runs fully offline. No React, no single-page app, no authentication. 99 automated tests cover content and behaviour; model-answer claims about code output are verified by running the code.

> **Limitations.** Activities and Block Model tags are the author's judgement, not yet validated with students or a second rater. Keyword assessment is coarse for code answers, and no AI assessor has been compared with human ratings (see `design_notes.md`).

Further detail: `design_notes.md` (design rationale, ethics, future work) and `README.md` (setup and configuration).